

8.5小组模拟面

1.auto和decltype比较

两个都是类型推导，但是decltype可以推导表达式，还可以推导函数返回值等，比如
`int a=0; double b=1.0; decltype(a+b) c=2.0;`

同时decltype推导的结果还包含了顶层的引用（也就是有const带const），比如
`const int a;` 如果用 `auto b=a;` b的类型是int，但是用 `decltype(a) b=a;` b的类型是const int

两个特殊情况：

- decltype如果推导**赋值表达式的类型**，返回是**等号左边类型的引用**，比如 `int i;`
`decltype (i=2) b=2;` 此时b的类型是int &

这里注意一下，`decltype (i=2) b=2;` 不能看作是 `int& b=2;` 因为2是纯右值，不能给左值引用赋值，这个语句应该看作这几步

`i=2;`

`int& b=i;`

`b=2;` 也就是先给i赋值，然后b看作是i的引用，最后给b赋值（同步修改i了）

所以如果是`decltype (i=2)b=3;` 最后i和b都是3

- decltype如果推导**指针类型的解引用**，返回是**对应类型的引用**，比如 `int *p=&a;`
`decltype( *p)` 结果类型是int &

2.你对智能指针的理解

首先智能指针是防止内存泄露的，使用RAII思想（对象创建即初始化（强调一次性获取和析构所有资源），对象回收即析构，无需中途手动管理），本质使用栈区资源管理堆区对象

3.基于RAII思想设计实现的功能，组件？

C++11的锁（`lock_guard`，`unique_lock`），智能指针，内存池

unique_ptr接口函数，底层实现

智能指针用 . 使用其本身提供的接口函数（接口函数如下），用 -> 使用原生指针对应类的函数

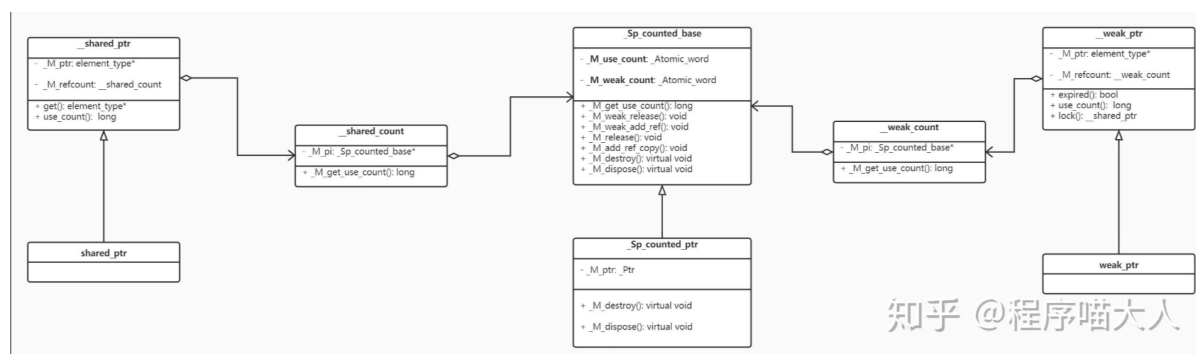
- get() 获取其保存的原生指针
- bool() 判断是否拥有指针
- release() 释放所管理指针的所有权，返回原生指针。但并不销毁原生指针。
- reset() 释放并销毁原生指针。如果参数为一个新指针，将管理这个新指针

```
1 std::unique_ptr<A> a1(new A());
2 A *origin_a = a1.get(); //尽量不要暴露原生指针
3 if(a1)
4 {
5     // a1 拥有指针
6 }
7
8 std::unique_ptr<A> a2(a1.release()); //常见用法，转义所有权
9 a2.reset(new A()); //释放并销毁原有对象，持有一个新对象
10 a2.reset(); //释放并销毁原有对象，等同于下面的写法
11 a2 = nullptr; //释放并销毁原有对象
```

unique_ptr的底层很简单，就是一个有两个参数的类模版，第一个参数是指针指向的空间，第二个参数是unique_ptr的析构器（这个参数是默认的，如果需要我们可以自己写），在内部禁用了各种拷贝，赋值，移动构造并实现了上述的几个接口

4.shared_ptr底层实现分析，接口函数

参考文档：[源码分析shared_ptr实现 - 知乎](#)



如图，shared_ptr实际上继承了__shared_ptr，里面有两个成员（只有一个重要，就只讲一个）

首先有个_shared_count类型的 _M_refcount, shared_count类型内部有 _Sp_counted_base类型的指针, 这个指针指向其子类 _Sp_counted_ptr (该子类中有一个_ptr类型的指针M_ptr, 指向shared_ptr共同维护的空间), 其父类 Sp_counted_base空间中维护两个原子变量M_use_count, 和M_weak_count, 分别维护强引用计数和弱引用计数, 当shared_ptr发生拷贝, 指向M_ptr对应空间时, 强引用计数增加, 销毁时减少, **为0时调用析构shared_ptr, 并调用管理对象的析构函数**。强引用计数增加时, 弱引用计数同步增加, **弱引用计数为0时回收M_ptr所指公共空间**

强引用计数归0, 对应对象都析构了, 为什么还保留空间, 等到弱引用技术归0再销毁? : 因为weak_ptr不单独创建, 必须依附shared_ptr, 此时二者共用控制块, 以及指向同一块空间, 如果直接销毁了, weak_ptr使用时就变成野指针, 起不到weak_ptr的作用

简单来说就是分为两部分, 一个是模板元编程创建的元指针, 第二个是一个控制块, 强引用计数和shared_ptr有关, 弱引用计数和weak_ptr有关, 还有一个删除器, 弱引用技术归0时释放公共地址空间

接口函数

功能分类	接口 / 方法	说明
构造	<code>shared_ptr<T>()</code>	默认构造，空指针
	<code>shared_ptr<T>(T*)</code>	不推荐，用 <code>make_shared</code> 更好
	<code>make_shared<T>(...)</code>	☑ 推荐，安全高效
赋值	拷贝构造 / 移动构造	控制引用计数
	拷贝赋值 / 移动赋值	管理权转移或共享
析构/重置	<code>reset()</code>	释放当前对象，ptr 变空
	<code>reset(T*)</code>	改为管理新对象
访问对象	<code>get()</code>	获取原始指针（只读，不管理生命周期）
	<code>operator*</code> , <code>operator-></code>	访问对象成员
	<code>operator bool()</code>	判断是否非空
引用计数	<code>use_count()</code>	返回当前引用数（调试用）
	<code>unique()</code>	(C++17 弃用) 是否唯一持有
弱指针	可赋值给 <code>std::weak_ptr<T></code>	用于观察，不增加引用计数

5.new直接构造和make_shared构造的区别

首先确定一点，`make_shared`只用于第一次对空间的分配，也就是第一个`shared_ptr`，而不是用于后续的移动构造或者拷贝构造把空间共享给其他的`shared_ptr`。

`make_shared`构造的好处在于：直接把控制块和对象内存一起分配了，且形成连续空间，如果用`new`，则分配两次（如下），且空间不连续

步骤	说明	分配了什么
1	<code>new Foo()</code>	分配对象内存 (比如 <code>sizeof(Foo)</code>) , 创建出 <code>Foo</code> 类型的对象
2	<code>std::shared_ptr<Foo></code> 构造函数	分配控制块 (control block) , 其中包含: ◆ 强引用计数 (<code>use_count</code>) ◆ 弱引用计数 (<code>weak_count</code>) ◆ 指向对象的指针 (<code>M_ptr</code>) ◆ 可能还有删除器、分配器等

6.如果不用信号量或控制块怎么实现shared_ptr

不用控制块如何实现 (实现MyShared_ptr) :可以在类内加一个vector (里面的成员是MyShared_ptr* 或MyShared_ptr&) , 当一个裸指针指向之前已经被指向的空间时, 加入b指向a, 那么就先用b的vector去拷贝a的一份, 然后加上自己的, 然后再从a的vector依次遍历里面的对象 (指针或引用都能找到) , 在每个对象的vector中都找位置加上b。如果删除的话也是类似 (最后删自己) 。注意到回收的时候, 可能会误操作删除野指针, 所以我们可以重载delete, 具体流程是先通过要删除的指针, 找到其vector中所有要删除的其他指针中自己的地址, 赋空, 最后删除自己并退出

7.weak_ptr应用场景, 底层, 接口函数

用于解决shared_ptr的循环指向问题, 或者一些特殊的场合, 我们只需要确定智能指针所指的對象是否存活 (如果用shared_ptr, 此时对象永远存活, **具体场景比如可以用在观察者模式中, 观察者只想知道被观察者是否存活, 可以保留被观察者的一个weak_ptr指针**) , 此时用weak_ptr

weak_ptr用弱引用计数, 不能单独指向 (维护) 一个对象空间 (必须依附已创建好的shared_ptr) , 和其共用一个控制块, 当需要获取其所指对象, 并具体调用相关操作时, 需要调用**.lock函数** (该函数会通过一个叫做enable_from_this的类, 查看控制块中强引用计数是否不为0, 是则表示空间还在, 此时会创建并返回一个shared_ptr指向该空间, 同时强弱引用计数都加一)

主要接口函数

功能	接口函数	说明
构造观察者	<code>weak_ptr(const shared_ptr<T>&)</code>	从 <code>shared_ptr</code> 构造，不增加引用计数
安全访问对象	<code>std::shared_ptr<T> lock() const</code>	如果对象存活，返回有效的 <code>shared_ptr</code> ，否则返回空
检查对象是否可能失效	<code>bool expired() const</code>	只是判断 <code>use_count</code> 是否为 0
拷贝 / 移动构造/赋值	<code>weak_ptr(const weak_ptr&)</code> 等	用于传递观察权，不影响对象
从 <code>shared_ptr</code> 赋值	<code>operator=(const shared_ptr<T>&)</code>	方便赋值观察对象

8.智能指针是线程安全的吗？

首先 `unique_ptr` 一定不安全（安全要 `shared_ptr` 干什么）

`shared_ptr` 和 `weak_ptr` 我们从底层代码可以看到，维护的强引用计数和弱引用计数都是原子操作，即和控制块有关的操作（比如同时多个 `shared_ptr` 同时构造，析构涉及的引用计数加减都是安全的），但是对于公共对象的操作不是线程安全的（这个要通过对应对象的接口函数进行手动控制并发）

9.lambda定义，应用场景，编译器如何编译lambda表达式

lambda 表达式 是一种匿名函数，可以无需声明或定义函数，原地构造出一个函数并使用（一般 `lambda` 表达式函数体不会太大）

应用场景：

- STL 算法、线程、回调等
- 捕获上下文中的变量
- 配合现代 C++ 特性（如智能指针、RAII、异步等）写出更灵活、更强大的代码

具体编译过程：lambda在编译的时候编译器会将其看作一个匿名对象（闭包对象），其中捕获的参数列表作为其成员（值捕获直接拷贝，地址捕获用引用或指针），内部函数重载了operator()，使其成为一个可调用对象

对前面的lambda表达式和std::move搭配使用会出错的补充：之前说到这两个不能同时使用，具体原因是因为我们从外部捕获的指针或引用，如果用std::move转交使用权，涉及到原有的变量指针指向空，而lambda表达式本身是闭包的，不允许做修改操作

10.你用lambda表达式出过bug吗？

- 值捕获和引用捕获一起，出现冲突
- 从外捕获指针变量，过了生命周期被销毁，导致悬浮指针
- move尝试转交指针所指空间，但忘了lambda本身是闭包

11.lambda表达式的[]有捕获变量的能力，如果不支持[]捕获外部变量，应该怎么实现类似功能

- 把需要的参数按照参数传入，或者全局变量，静态变量形式封装，或者封装成一个结构体作为参数传入
- 利用bind包装lambda表达式生成可调用对象

12.function，bind的定义和使用

std::function是一个可调用对象包装器，是一个类模板，可以容纳除了类成员函数指针之外的所有可调用对象，它可以用统一的方式处理函数、函数对象、函数指针，并允许保存和延迟它们的执行。

- std::function对C++中各种可调用实体(普通函数、Lambda表达式、函数指针、以及其它函数对象等)的封装，形成一个新的可调用的std::function对象，简化调用；
- std::function对象是对C++中现有的可调用实体的一种**类型安全的包裹**（如：函数指针这类可调用实体，是类型不安全的）。

什么叫类型安全，就是避免隐式类型转换，有相应的类型检查

std::bind可以看作一个通用的**函数适配器**，它接受一个可调用对象，生成一个新的可调用对象来适应原对象的参数列表。（**注意bind是生成可调用对象的，和lambda一样**）

std::bind将可调用对象与其参数一起进行绑定，绑定后的结果可以使用std::function保存。std::bind主要有以下两个作用：

- 将可调用对象和其参数绑定成一个仿函数；
- 只绑定部分参数，减少可调用对象传入的参数。

13.tcp第三次握手丢失会发生什么，为什么ack没设计重传

客户端收到服务端的 SYN-ACK 报文后，就会给服务端回一个 ACK 报文，也就是第三次握手，此时客户端状态进入到 `ESTABLISH` 状态。

因为这个第三次握手的 ACK 是对第二次握手的 SYN 的确认报文，所以当第三次握手丢失了，如果服务端那一方迟迟收不到这个确认报文，就会触发超时重传机制，重传 SYN-ACK 报文，直到收到第三次握手，或者达到最大重传次数。

注意，**ACK 报文是不会有重传的，当 ACK 丢失了，就由对方重传对应的报文。**

第三次握手丢失，服务器会认为自己的第二次握手没有送达，于是会反复重传第二次握手报文，直到收到第三次握手报文

不用ack：一方面三次握手保证了服务器始终知道连接是否建立，如果没有建立客户端发包会被RST掉，另一方面没有必要为了ack再发一次ack，否则会陷入自证危机

14.TIME_WAIT的等待时间为什么是2MSL而非1MSL?

MSL即网络包最大生存时间，一来一回两次传输经过2MSL必定会消失，可以排除旧连接传输网络包问题

同时RST报文不会丢失，假如开启了端口复用，最后一步TIME_WAIT端口会立马拿来建立新连接，端口号ip都是一致的，此时RST在网络中如果延迟，到了之后就会把新的连接断开

15.进程间通信方式

16.RTMP和HLS协议的比较